
FreeToken-MLX: Memory-Bounded MoE Inference on Apple Silicon

Anonymous Author(s)

Abstract

MoE models activate few parameters per token but retain an expert pool that can exceed edge-device memory. We present FREETOKEN-MLX, an Apple Silicon backend inside FreeToken that preserves its cross-layer expert bank, cache, eviction, and offload path while executing selected experts with MLX. It chooses resident MLX when safe and bounded offload otherwise, and can quantize only routed experts. On one 16GB Apple M4, stock BF16 MLX failed before token one; bounded BF16 offload generated at 1.88 token/s. Expert-only Q4 reached 4.49 token/s at 7.93GB peak. Under an equal cache-byte budget, Q3 gate/up and Q4 down improved 128-token throughput by 11.1% over uniform Q4. This single-device study shows a memory-throughput Pareto path, not universal superiority over resident MLX.

1 Motivation and contribution

Sparse activation reduces arithmetic but not expert storage. Prior systems page experts on personal devices [5, 1, 7]; FreeToken instead builds a shared elastic cache and adapts execution to hardware bandwidth [6]. Its release primarily targets CUDA. Apple Silicon instead exposes unified CPU/GPU memory through MLX [2]; eagerly materializing a large checkpoint can still exceed the safe working set and terminate inference or displace other applications.

FREETOKEN-MLX adapts FreeToken rather than building an independent engine. It contributes: (1) an MLX backend reusing the expert bank, global cache, counters, and eviction; (2) safe resident/offload selection with explicit headroom; and (3) routed-expert-only, projection-aware precision. The scope is batch-one Qwen1.5-MoE-A2.7B on Apple Silicon.

2 System design

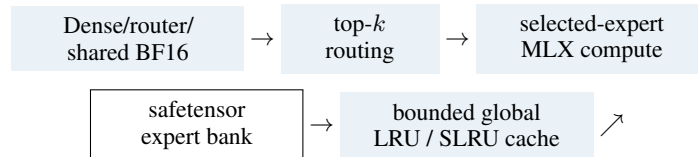


Figure 1: Only router-selected experts enter the MLX compute graph; FreeToken’s expert bank and bounded cache remain on the serving path.

Execution and caching. Dense attention, router, and shared-expert tensors remain resident MLX arrays. Routed tensors remain separate safetensors entries addressed by (layer, expert). The router synchronizes only its small top- k index tensor. Each selection performs a FreeToken cache lookup: a hit reuses evaluated MLX arrays; a miss materializes the expert bank entry and may evict a previous

Table 1: Measured feasibility and equal-budget precision ablation. *Peak reported by MLX before Metal OOM.

Path	Tokens	Slots	Misses	tok/s	Peak
Stock MLX-LM BF16	0	—	—	—	28.63GB*
FreeToken BF16 offload	32	121	—	1.88	5.83GB
BF16 dense / Q4 experts	32	864	—	4.49	7.93GB
Uniform Q4 experts	128	864	3,489	7.31	8.02GB
Q3 gate/up, Q4 down	128	1,015	2,822	8.12	8.03GB

expert. Periodic graph evaluation bounds lazy references that would otherwise keep evicted weights alive. Logs expose real hits, misses, and evictions.

Memory safety and hybrid selection. The runtime estimates resident peak memory from checkpoint, packing, runtime, and optional draft-model bytes. It selects resident MLX only within both the MLX limit and physical memory minus requested headroom under current pressure. Otherwise it budgets the expert cache after dense/runtime reserves. If one expert cannot fit, startup fails; cache shrinkage is monotonic per request.

Expert precision. The converter streams BF16 shards, preserves non-routed weights, and applies MLX affine packing only to routed projections. Uniform Q4 is default; opt-in Q3 gate/up with Q4 down reduces routed-expert storage about 16.7%, admitting more experts per byte. This is a layout integration, not a new quantizer; mixed-precision offloading has precedent [4].

3 Evaluation

We evaluate Qwen1.5-MoE-A2.7B [3] (24 layers, 60 routed experts/layer, top-4), batch one, on an Apple M4 with 16GB unified memory, macOS 15.3, Python 3.12.13, and MLX-LM 0.31.3. Generation is greedy from Hello. Safety-sensitive runs cap MLX at 10GB and reserve 3.2GB of system headroom. Peaks are MLX allocator measurements rather than total macOS memory. The first three rows in Table 1 use different token counts and show deployment feasibility; the final two form the controlled comparison.

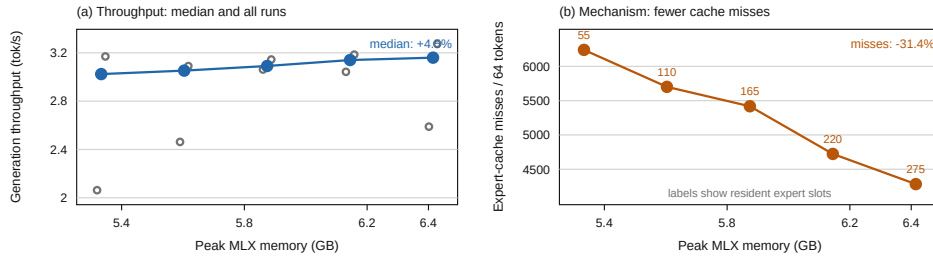


Figure 2: Q4 cache sweep (64 tokens, $n = 3$ fresh processes/point, 45s cooldown, rotating order): all throughput observations and medians; misses are labelled by slots. Moving 55–275 slots raised peak 5.33–6.42GB, cut misses 31.4%, and raised median throughput 4.5%; overlapping ranges limit the claim.

Capacity and throughput. Stock `mlx-lm` attempted to materialize BF16, reported 28.63GB, and failed before token one. Bounded FreeToken offload completed 32 tokens at 1.88 token/s and 5.83GB. This path adds capacity, but expert reads remain the main bottleneck. Quantizing routed experts alone reached 4.49 token/s at 7.93GB, 2.39 $\times$ the reported BF16-offload rate while retaining dense, attention, router, and shared weights in BF16. It changes routed-expert numerics and is therefore a separate Pareto point, not a lossless speedup.

Controlled projection ablation. With fixed pressure, a 10GB limit, 3.2GB headroom, prompt, and cache bytes, we interleaved two fresh runs per layout. Projection-aware precision admitted

53 1,015 rather than 864 experts, cut misses 19.1%, and reduced paired mean time from 17.52s to
54 15.77s (7.31 to 8.12 token/s, +11.1%) at nearly equal peak. Forced to 864 slots, it remained 3.4%
55 faster and used 0.62GB less peak memory.

56 **Quality proxy and negative evidence.** A 135-token proxy yielded NLL 2.618 (BF16), 2.707
57 (Q4), 2.725 (Q3/Q3/Q4), and 2.865 (Q3). It cannot establish quality, so Q4 remains default; expert
58 stacking and a grouped-Q4 kernel both slowed end-to-end execution.

59 References

- 60 [1] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with
61 offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- 62 [2] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and
63 flexible machine learning on apple silicon, 2023. URL [https://github.com/ml-explore/](https://github.com/ml-explore/mlx)
64 [mlx](https://github.com/ml-explore/mlx).
- 65 [3] Qwen Team. Qwen1.5-moe-a2.7b model card. [https://huggingface.co/Qwen/Qwen1.](https://huggingface.co/Qwen/Qwen1.5-MoE-A2.7B)
66 [5-MoE-A2.7B](https://huggingface.co/Qwen/Qwen1.5-MoE-A2.7B), 2024.
- 67 [4] Peng Tang et al. Hobbit: A mixed precision expert offloading system for fast moe inference.
68 *arXiv preprint arXiv:2411.01433*, 2024.
- 69 [5] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Efficient moe infer-
70 ence on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*,
71 2024.
- 72 [6] Shuo Yang, Xiaoze Fan, Melissa Pan, Haocheng Xi, Zhe Wang, Shanlin Sun, Kurt Keutzer,
73 Song Han, Matei Zaharia, Chenfeng Xu, and Ion Stoica. Freetoken: Efficient edge-native moe
74 serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.
- 75 [7] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. Edgemoe:
76 Fast on-device inference of moe-based large language models. *arXiv preprint arXiv:2308.14352*,
77 2023.